Deployment of Deep Learning Architectures in Docker Containers

ViT image classification with FastAPI, Gradio and Docker Compose

Diego Arnaldo Martínez Puertas

AIIA — October 2026

Source code: `https://github.com/Diegomartinezpuertas/vit-docker-deployment`

Contents

1 Selected model **2**

2 Architecture **2**

3 Inference image **2**

3.1 API 2

3.2 Dockerfile 2

3.3 Build cache 3

4 Running the inference container **3**

5 Docker Compose and graphical inference **4**

6 Extensions **6**

6.1 Fine-tuning on the GPU 6

6.2 Other improvements 6

7 Problems found and conclusions **6**

1 Selected model

I used **ViT-Base/16** (`google/vit-base-patch16-224`), a Vision Transformer that splits a 224×224 image into 16×16 patches and processes them with a 12-layer Transformer encoder (86M parameters). The checkpoint is pre-trained on ImageNet-21k and fine-tuned on ImageNet-1k (1000 classes). I chose it because it is small enough for an 8 GB laptop GPU, has a clean `transformers` API and is easy to fine-tune for the optional part. The stack is PyTorch 2.7.1 with CUDA 12.8, running on WSL2 with an NVIDIA RTX 5070 Laptop GPU.

2 Architecture

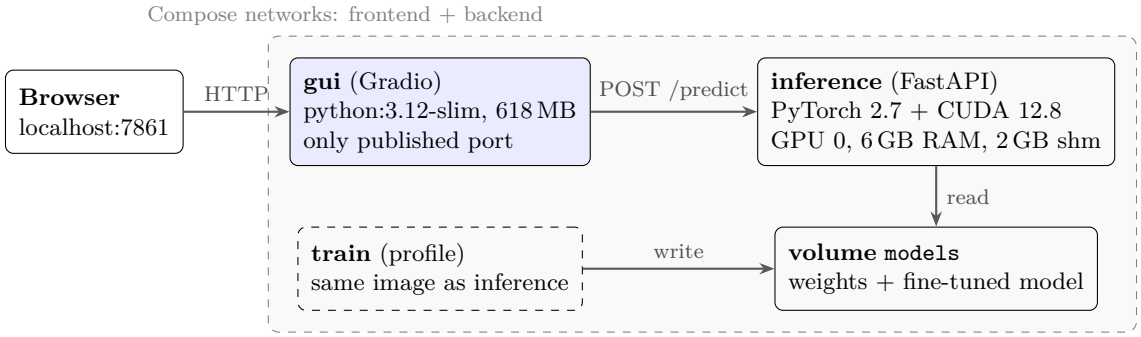


Figure 1: Services defined in `compose.yaml`. Only the GUI is published to the host.

The inference container only runs the model behind a REST API; the GUI container only renders the interface and calls the API by its service name (`http://inference:8000`). The GUI is attached to a `frontend` and a `backend` network, the API only to `backend`, and port 8000 is not published.

3 Inference image

3.1 API

`inference/app.py` loads the processor and the model once, in FastAPI's `lifespan` handler, moves the model to `cuda` if available (CPU otherwise) and calls `eval()`. `POST /predict` receives an image as multipart data, preprocesses it, runs the forward pass inside `torch.inference_mode()` and returns the top-5 labels with their softmax score. `GET /health` reports the model and device and is used by the healthcheck. The endpoint is a plain `def`, so FastAPI runs it in a thread pool and the GPU work does not block the event loop. The served model is chosen with the `MODEL_ID` environment variable, so the image does not need to be rebuilt to switch models.

3.2 Dockerfile

`inference/Dockerfile` (comments removed):

```
FROM pytorch/pytorch:2.7.1-cuda12.8-cudnn9-runtime
ENV DEBIAN_FRONTEND=noninteractive \
    PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_DISABLE_PIP_VERSION_CHECK=1 \
    HF_HOME=/models
RUN apt-get update && apt-get install -y --no-install-recommends curl && rm -rf /var/lib/apt/lists/*
RUN useradd --create-home --uid 1000 user && mkdir /models && chown user:user /models
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY --chown=user:user . .
USER user
EXPOSE 8000
```

```
HEALTHCHECK --interval=30s --timeout=5s --start-period=60s --retries=3 \
    CMD curl -f http://localhost:8000/health || exit 1
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Design decisions:

- **Base image.** The RTX 5070 (Blackwell, `sm_120`) needs PyTorch ≥2.7 built with CUDA ≥12.8; older images fail with *no kernel image is available for execution on the device*. Using the official image avoids installing PyTorch with pip, so `torch` is not in `requirements.txt`.
- **Layer order.** Rarely changing steps first (system packages, dependencies), source code last. Only `requirements.txt` is copied before `pip install`.
- **Size.** `apt-get` install and clean-up in the same `RUN`, `--no-install-recommends`, `pip --no-cache-dir` and a `.dockerignore`. Versions pinned from `pip freeze`.
- **Non-root user.** `/models` is created and owned by the user, so a new named volume mounted there inherits the ownership.
- **Healthcheck.** `curl -f` fails on HTTP errors; the 60 s start period covers model loading.

3.3 Build cache

Figure 2 measures a rebuild after editing a comment in `app.py`, with the code copied after or before the dependency install. With the wrong order every code change reinstalls all dependencies; here it costs 11.8 s instead of 0.7 s, but it would be minutes if PyTorch were installed with pip.

Dockerfile layer order	Rebuild time after editing app.py	Layers rebuilt
requirements.txt → pip install → COPY . . (cache-friendly, used in the project)	**0.7 s**	Only COPY . . (0.1 s) pip install: CACHED
COPY . . → pip install (code copied before dependencies)	**11.8 s**	COPY . . and pip install (6.7 s)

Rebuild after adding a comment to app.py, both caches warm. Image: pytorch/pytorch:2.7.1-cuda12.8-cudnn9-runtime (WSL2, Docker 29.6).

Figure 2: Rebuild time depending on the layer order.

`docker images` reports 12.6 GB for the final image. About 12.2 GB of it is the base image, whose largest layer (Conda + PyTorch) is 7.65 GB in `docker history`; my own layers add about 0.45 GB, almost all of it the `pip install` layer (448 MB).

4 Running the inference container

```
docker build -t vit-inference:1.0 ./inference
docker run -d --name vit-inference --gpus '"device=0"' \
  --memory=6g --memory-swap=6g --cpus=4 --shm-size=2g \
  -p 8000:8000 -v hf-cache:/models -e MODEL_ID=google/vit-base-patch16-224 \
  --restart unless-stopped vit-inference:1.0
curl -F "file=@cat.jpg" http://localhost:8000/predict
```

Parameter	Purpose
`--gpus '"device=0"'`	Expose only GPU 0 instead of all devices.
`--memory=6g --memory-swap=6g`	RAM cap with no extra swap; above it the container is OOM-killed (exit 137).
`--cpus=4`	Limit CPU usage to 4 cores.
`--shm-size=2g`	`/dev/shm` for PyTorch DataLoader workers (default is 64 MB, see Sec. 6.1).
`-v hf-cache:/models`	Persist downloaded weights between containers.
`-e MODEL_ID`	Select the model without rebuilding.
`--restart unless-stopped`	Restart automatically if the process dies.

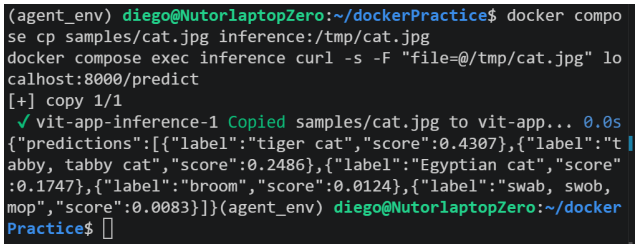


Figure 3: Terminal inference, run inside the inference container of the Compose stack (same image, `docker compose cp`/`exec`): `tiger cat` 0.43, `tabby cat` 0.25, `Egyptian cat` 0.17.

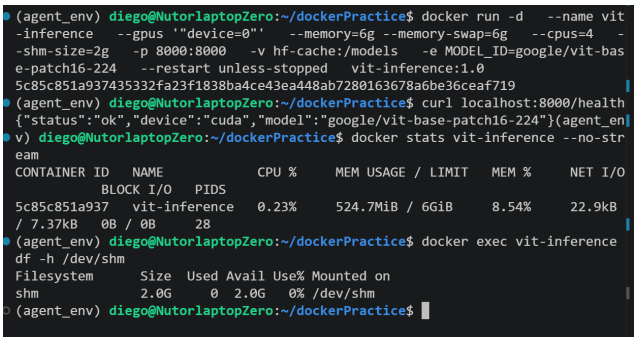


Figure 4: The `docker run` above: `/health` reports `cuda`, `docker stats` the 6 GiB limit and `df` the 2 GB `/dev/shm`.

5 Docker Compose and graphical inference

`compose.yaml` (complete file):

```
name: vit-app
# Reusable blocks (YAML anchors): GPU + limits, and log rotation.
x-gpu: &gpu
  deploy:
    resources:
      limits: {memory: "${INFERENCE_MEMORY:-6g}", cpus: "${INFERENCE_CPUS:-4}"}
      reservations:
        devices: [{driver: nvidia, device_ids: ["${GPU_ID:-0}"], capabilities: [gpu]}]
x-logging: &logging
  logging: {driver: json-file, options: {max-size: "10m", max-file: "3"}}
services:
  inference: # FastAPI + PyTorch on the GPU, not published to the host
    build: ./inference
    image: vit-inference:1.0
    <<: [*gpu, *logging]
    environment: {MODEL_ID: "${MODEL_ID:-google/vit-base-patch16-224}"}
    volumes: [models:/models]
    shm_size: "2gb"
    expose: ["8000"]
    restart: unless-stopped
    networks: [backend]
  gui: # Gradio, the only service with a published port
    build: ./gui
    image: vit-gui:1.0
    <<: *logging
    ports: ["${GUI_PORT:-7860}:7860"]
    environment:
      API_URL: http://inference:8000
      MODEL_ID: "${MODEL_ID:-google/vit-base-patch16-224}" # unused; recreates the GUI when the model changes
    depends_on: {inference: {condition: service_healthy}}
    deploy: {resources: {limits: {memory: 1g, cpus: "1"}}}
    restart: unless-stopped
    networks: [frontend, backend]
```

```
  train: # one-off fine-tuning job: docker compose --profile train run --rm train
    build: ./inference
    image: vit-inference:1.0
    profiles: [train]
    <<: [*gpu, *logging]
    command: python train.py --epochs ${EPOCHS:-3} --output /models/vit-beans
    volumes: [models:/models]
    shm_size: "2gb"
    ulimits: {memlock: -1, stack: 67108864} # recommended by NVIDIA for PyTorch
    networks: [backend]
volumes:
  models:
networks:
  frontend:
  backend:
```

`gui/Dockerfile` (comments removed):

```
FROM python:3.12-slim
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_DISABLE_PIP_VERSION_CHECK=1 \
    API_URL=http://inference:8000
RUN useradd --create-home --uid 1000 user
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY --chown=user:user . .
USER user
EXPOSE 7860
HEALTHCHECK --interval=30s --timeout=5s --start-period=20s --retries=3 \
    CMD python -c "import␣urllib.request;␣urllib.request.urlopen('http://localhost:7860/',␣timeout=4)" || exit 1
CMD ["python", "app.py"]
```

The GUI image needs no PyTorch: `python:3.12-slim` with `gradio` and `requests` is 618 MB instead of 12.6 GB, and its healthcheck uses Python's `urllib`, so no `apt-get` layer is needed. The values in `${...}` come from `.env`; the host port is 7861 because 7860 was already in use on my machine.

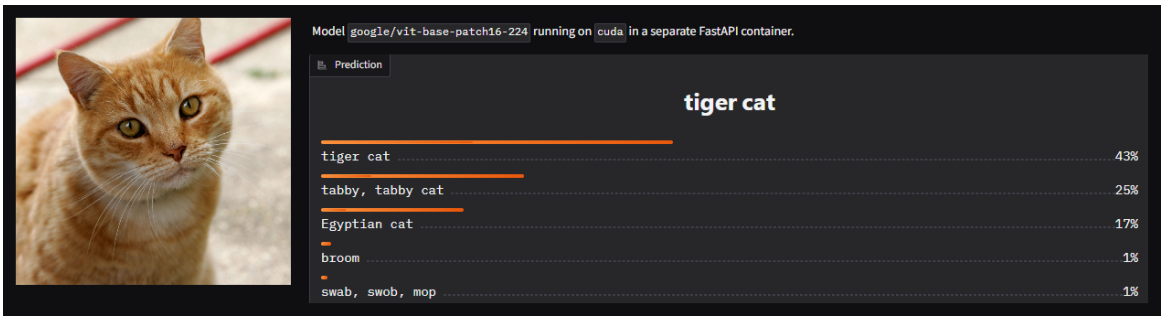


Figure 5: Graphical inference through the Compose stack (`localhost:7861`): input image and the top-5 returned by the API, which reports the model and `cuda` as device.

Isolation check: `curl localhost:8000` from the host is refused, while `docker compose exec inference curl localhost:8000/health` answers, and `docker compose ps` shows both services (`healthy`) with only `7861->7860` published.

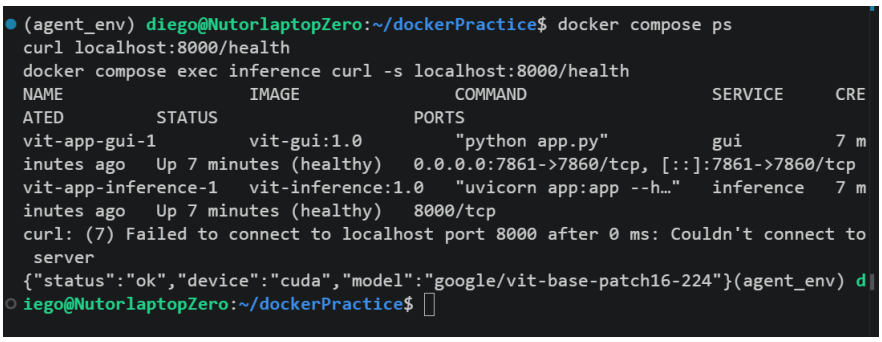


Figure 6: Running stack: only the GUI port is exposed.

6 Extensions

6.1 Fine-tuning on the GPU

`train.py` replaces the 1000-class head with a 3-class head and fine-tunes ViT on the `beans` dataset (1034 train / 133 validation / 128 test leaf images) with AdamW ($lr = 2 \cdot 10^{-5}$, batch 32, bf16 autocast, 4 DataLoader workers). It runs with `docker compose --profile train run --rm train` and saves the model in the shared volume; setting `MODEL_ID=/models/vit-beans` in `.env` makes the API serve it. Table 1 shows the training run.

Table 1: Fine-tuning ViT on `beans` (RTX 5070, bf16). Epoch 0 is the new 3-class head before training.

Epoch	Train loss	Val. accuracy	Time (s)
0 (new head)	–	0.414	–
1	0.521	0.955	7.5
2	0.106	0.977	6.1
3	0.030	0.977	6.1

Test accuracy is **0.977**, with a peak of 3.4 GiB of GPU memory. Running the same job without `--shm-size` (64 MB default) fails with *DataLoader worker is killed by signal: Bus error*, which is why both GPU services set `shm_size: 2gb`. The training service also sets the `memlock` and `stack` ulimits recommended by NVIDIA for PyTorch containers.

6.2 Other improvements

- Non-root users (UID 1000) in both images.
- Healthchecks, and `depends_on` waits for a healthy API.
- API only on the `backend` network, port not published.
- Resource limits for every service, GUI included (1 GB, 1 CPU).
- Log rotation (`json-file`, 10 MB × 3).
- Settings in `.env`; YAML anchors for repeated blocks.
- One image for API and training (`profiles`); `MODEL_ID` picks the model.
- Pinned image tag and dependency versions, `.dockerignore`.
- `gui/client.py`: calls the GUI as an API with `gradio_client` (endpoint `/predict`).

```
docker compose run --rm --no-deps -v ./samples:/samples gui python client.py http://gui:7860 /samples/cat.jpg
```

7 Problems found and conclusions

- **Blackwell GPU:** needed a CUDA 12.8 base image, which also fixed the PyTorch version.
- **Layer order:** code copied before the dependencies made every edit reinstall them (Fig. 2).

- `localhost` inside a container is the container itself: the GUI reaches the API by service name.
- **Shared memory:** DataLoader workers crash with the default 64 MB `/dev/shm`.
- **Port conflict:** 7860 was taken by another container; the host port moved to `.env`.

Splitting model and interface keeps the GPU image heavy but stable and the GUI light and quick to rebuild, and the same inference image serves both the API and the training job. Most of the work was not the model itself but matching CUDA versions, layer ordering and resource limits.